

4.1 Perceptron.h

```
#ifndef PERCEPTRON_HPP
#define PERCEPTRON_HPP

#include <iostream>

template <size_t Samples, size_t Inputs, size_t Outputs>
class Perceptron {
private:
    int** X;

    int** Y;

    double W[Outputs][Inputs]{};

    double b[Outputs]{};

    int Epocas = 10;

    int activation(double val) {
        return (val > 0) ? 1 : 0;
    }

public:
    Perceptron(int** entradas, int** salidas) {
        X = new int* [Samples];

        for (size_t i = 0; i < Samples; ++i) {
            X[i] = new int[Inputs];

            for (size_t j = 0; j < Inputs; ++j)
```

```
        X[i][j] = entradas[i][j];  
    }
```

```
Y = new int* [Samples];  
for (size_t i = 0; i < Samples; ++i) {  
    Y[i] = new int[Outputs];  
    for (size_t j = 0; j < Outputs; ++j)  
        Y[i][j] = salidas[i][j];  
}
```

```
for (int o = 0; o < Outputs; ++o) {  
    for (int j = 0; j < Inputs; ++j)  
        W[o][j] = 0.5;  
    b[o] = -2.0;  
}  
}
```

```
~Perceptron() {  
    for (size_t i = 0; i < Samples; ++i) {  
        delete[] X[i];  
        delete[] Y[i];  
    }  
    delete[] X;  
    delete[] Y;  
}
```

```

void Entrenar() {
    for (int epoca = 0; epoca < Epocas; ++epoca) {
        bool errorTotal = false;

        for (int i = 0; i < Samples; ++i) {
            for (int o = 0; o < Outputs; ++o) {
                double l = b[o];

                for (int j = 0; j < Inputs; ++j)
                    l += W[o][j] * X[i][j];

                int y_pred = activation(l);
                int e = Y[i][o] - y_pred;

                if (e != 0) errorTotal = true;

                for (int j = 0; j < Inputs; ++j)
                    W[o][j] += e * X[i][j];
                b[o] += e;
            }
        }

        if (!errorTotal) break;
    }
}

```

```

void Pesos() {

```

```

using namespace std;

for (int o = 0; o < Outputs; ++o) {
    cout << "Salida " << o << " - Pesos: ";
    for (int j = 0; j < Inputs; ++j)
        cout << W[o][j] << "\t";
    cout << "Bias: " << b[o] << endl;
}
}

```

```

void Evaluar() {
    using namespace std;
    cout << "\nEvaluacion de salidas:\n";
    for (int i = 0; i < Samples; ++i) {
        cout << "Entrada " << i << ": ";
        for (int j = 0; j < Inputs; ++j)
            cout << X[i][j] << " ";
        cout << "-> Salida: ";
        for (int o = 0; o < Outputs; ++o) {
            double l = b[o];
            for (int j = 0; j < Inputs; ++j)
                l += W[o][j] * X[i][j];
            cout << activation(l);
        }
        cout << endl;
    }
}
}

```

```
};
```

```
#endif
```

4.2 Red_Neuronal_m.cpp

```
#include "Perceptron.h"
```

```
int main() {
```

```
    const int Samples = 12;
```

```
    const int Inputs = 35;
```

```
    const int Outputs = 4;
```

```
    int datos[Samples][Inputs] = {
```

```
        {0,1,1,1,0,1,0,0,0,1,1,0,0,0,1,1,0,0,0,0, 1,0,0,0,1,0,1,1,1,0}, // 0
```

```
        {0,0,1,0,0,0,1,1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,1,1,0}, // 1
```

```
        {0,1,1,1,0,1,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,1,0,0,0,1,1,1,1}, // 2
```

```
        {0,1,1,1,0,1,0,0,0,1,0,0,0,0,1,0,0,1,1,0,0,0,0,0,1,1,0,0,0,1,0,1,1,1,0}, // 3
```

```
        {1,0,0,1,0,1,0,0,1,0,1,0,0,1,0,1,1,1,1,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0}, // 4
```

```
        {1,1,1,1,1,1,0,0,0,0,1,0,0,0,0,0,1,1,1,0,0,0,0,0,1,1,0,0,0,1,0,1,1,1,0}, // 5
```

```
        {0,1,1,1,0,1,0,0,0,0,1,0,0,0,0,0,1,1,1,0,1,0,0,0,1,1,0,0,0,1,0,1,1,1,0}, // 6
```

```
        {1,1,1,1,1,0,0,0,1,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,0,0}, // 7
```

```
        {0,1,1,1,0,1,0,0,0,1,1,0,0,0,1,0,1,1,1,0,1,0,0,0,1,1,0,0,0,1,0,1,1,1,0}, // 8
```

```
        {0,1,1,1,0,1,0,0,0,1,1,0,0,0,1,0,1,1,1,0,0,0,0,0,1,1,0,0,0,1,0,1,1,1,0}, // 9
```

```
        {0,1,1,1,0,1,0,0,0,1,1,0,0,0,1,0,1,1,1,0,1,0,0,0,1,1,0,0,0,1,1,0,0,0,1}, // A
```

```
        {1,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1,1,1,1,0,1,0,0,0,1,1,0,0,0,1,1,1,1,1,0} // b
```

```
};
```

```
int etiquetas[Samples][Outputs] = {  
    {0, 0, 0, 0}, // 0  
    {0, 0, 0, 1}, // 1  
    {0, 0, 1, 0}, // 2  
    {0, 0, 1, 1}, // 3  
    {0, 1, 0, 0}, // 4  
    {0, 1, 0, 1}, // 5  
    {0, 1, 1, 0}, // 6  
    {0, 1, 1, 1}, // 7  
    {1, 0, 0, 0}, // 8  
    {1, 0, 0, 1}, // 9  
    {1, 0, 1, 0}, // 10  
    {1, 0, 1, 1} // 11  
};
```

```
int** X = new int* [Samples];  
for (int i = 0; i < Samples; ++i) {  
    X[i] = new int[Inputs];  
    for (int j = 0; j < Inputs; ++j)  
        X[i][j] = datos[i][j];  
}
```

```
int** Y = new int* [Samples];  
for (int i = 0; i < Samples; ++i) {  
    Y[i] = new int[Outputs];
```

```

        for (int j = 0; j < Outputs; ++j)
            Y[i][j] = etiquetas[i][j];
    }

    Perceptron<Samples, Inputs, Outputs> p(X, Y);

    p.Entrenar();

    p.Pesos();

    p.Evaluar();

    for (int i = 0; i < Samples; ++i) {
        delete[] X[i];
        delete[] Y[i];
    }

    delete[] X;
    delete[] Y;

    return 0;
}

```